

$$L = \log_2(N + 1)$$

Thus, the time needed to carry out the common tree operations is proportional to the base 2 log of N. In Big O notation, you say such operations take $O(\log N)$ time.

If the tree isn't full or balanced, the analysis is difficult. You can say that for a tree with a given number of levels, average search times will be shorter for the nonfull tree than the full tree because fewer searches will proceed to lower levels.

Compare the tree to the other data storage structures we've discussed so far. In an unordered array or a linked list containing 1,000,000 items, finding the item you want takes, on average, 500,000 comparisons, basically $O(N)$. In a balanced tree of 1,000,000 items, only 20 (or fewer) comparisons are required because it's $O(\log N)$.

In an ordered array, you can find an item equally quickly, but inserting an item requires, on average, moving 500,000 items. Inserting an item in a tree with 1,000,000 items requires 20 or fewer comparisons, plus a small amount of time to connect the item. The extra time is constant and doesn't depend on the number of items.

Similarly, deleting an item from a 1,000,000-item array requires moving an average of 500,000 items, while deleting an item from a 1,000,000-node tree requires 20 or fewer comparisons to find the item, plus a few more comparisons to find its successor, plus a short time to disconnect the item and connect its successor. Because the successor is somewhere lower in the tree than the node to delete, the total number of comparisons to find both the node and its successor will be 20 or fewer.

Thus, a tree provides high efficiency for all the common data storage operations: searches, insertions, and deletions. Traversing is not as fast as the other operations, but it must be $O(N)$ to cover all N items, by definition. In all the data structures you've seen, it has been $O(N)$, but we show some other data structures later where it could be greater. There is a little more memory needed for traversing a tree compared to arrays or lists because you need to store the recursive calls or use a stack. That memory will be $O(\log N)$. That contrasts with the arrays and lists that need only $O(1)$ memory during traversal.

Trees Represented as Arrays

Up to now, we've represented the binary tree nodes using objects with references for the left and right children. There's a completely different way to represent a tree: with an array.

In the array approach, the nodes are stored in an array and are not linked by references. The position of the node in the array corresponds to its position in the tree. We put the root node at index 0. The root's left child is placed at index 1, and its right child at index 2, and so on, progressing from left to right along each level of the tree. This approach is shown in [Figure 8-24](#), which is a binary search tree with letters for the keys.

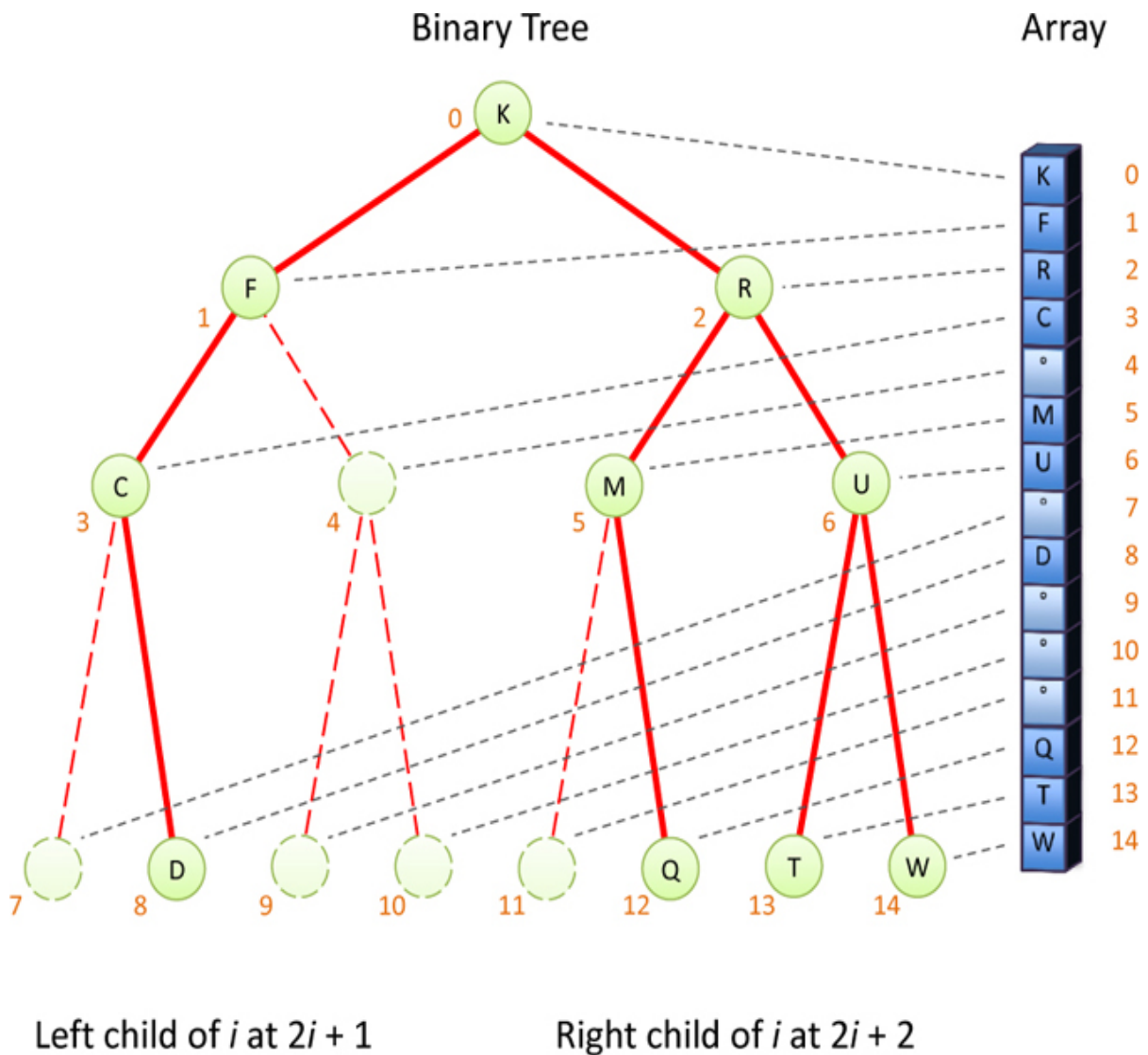


Figure 8-24 *A binary tree represented by an array*

Every position in the tree, whether it represents an existing node or not, corresponds to a cell in the array. Adding a node at a given position in the tree means inserting the node into the equivalent cell in the array. Cells representing tree positions with no nodes are filled with 0, `None`, or some other special value that cannot be confused with a node. In the figure, the $\circ$ symbol is used in the array for empty nodes.

With this scheme, a node's children and parent can be found by applying some simple arithmetic to the node's index number in the array. If a node's index number is `index`, this node's left child is

$$2 * \text{index} + 1$$

its right child is

$$2 * \text{index} + 2$$

and its parent is

$$(\text{index} - 1) // 2$$

(where the `//` indicates integer division with no remainder). You can verify these formulas work by looking at the indices in [Figure 8-24](#). Any algorithm that follows links between nodes can easily determine where to check for the next node. The scheme works for any binary tree, not just binary search trees. It has the nice feature that links between nodes are just as easy to travel up as they are going down (without the double linking needed for lists). Even better, it can be generalized to any tree with a fixed number of children.

In most situations, however, representing a tree with an array isn't very efficient. Unfilled nodes leave holes in the array, wasting memory. Even worse, when deletion of a node involves moving subtrees, every node in the subtree must be moved to its new location in the array, which is time-consuming in large trees. For insertions that insert nodes beyond the current maximum depth of the tree, the array may need to be resized.

If deletions aren't allowed or are very rare and the maximum depth of the tree can be predicted, the array representation may be useful, especially if obtaining memory for each node dynamically is, for some reason, too time-consuming. That might be the case when programming in assembly language or a very limited operating system, or a system with no garbage collection.

Tree Levels and Size

When trees are represented as arrays, the maximum level and number of nodes is constrained by the size of the array. For linked trees, there's no specific maximum. For both representations, the current maximum level and number of nodes can be determined only by traversing the tree. If there will be frequent calls to request these metrics, the `BinarySearchTree` object can maintain values for them, but the `insert()` and `delete()` methods must be modified to update the values as nodes are added and removed.

To count nodes in a linked tree, you can use the `traverse()` method to iterate over all the nodes and increment a count, as shown earlier in the example to find the average key value and again in the `nodes()` method of [Listing 8-10](#). To find the maximum level, you cannot use the same technique because the level of each node during the traversal is not provided (although it could be added by modifying the generator). Instead, the recursive definition shown in [Listing 8-10](#) gets the job done in a few lines of code.

Listing 8-10 *The `levels()` and `nodes()` Methods of `BinarySearchTree`*

```
class BinarySearchTree(object):    # A binary search tree class
...
    def levels(self):              # Count the levels in the tree
        return self.__levels(self.__root) # Count starting at root

    def __levels(self, node):      # Recursively count levels in subtree
        if node:                  # If a node is provided, then level is 1
            return 1 + max(self.__levels(node.leftChild), # more than
                           self.__levels(node.rightChild)) # max child
        else: return 0            # Empty subtree has no levels

    def nodes(self):              # Count the tree nodes, using iterator
        count = 0                 # Assume an empty tree
        for key, data in self.traverse(): # Iterate over all keys in any
            count += 1             # order and increment count
        return count
```

Counting the levels of a subtree is somewhat different than what you've seen before in that each node takes the maximum level of each of its subtrees and adds one to it for the node itself. It might seem as if there should be a shortcut by looking at the depth of the minimum or maximum key so that you don't

need to visit every node. If you think about it, however, even finding the minimum and maximum keys shows the depth only on the left and right “flanks” of the tree. There could be longer paths somewhere in the middle, and the only way to find them is to visit all the nodes.

Printing Trees

You’ve seen how to traverse trees in different orders. You could always use the traversal to print all the nodes in the tree, as shown in the visualization tool. Using the in-order traversal would show the items in increasing order of their keys. On a two-dimensional output, you could use the in-order sequence to position the nodes along the horizontal axis and the level of each node to determine its vertical position. That could produce tree diagrams like the ones shown in the previous figures.

On a simple command-line output, it’s easier to print one node per line. The problem then becomes positioning the node on the line to indicate the shape of the tree. If you want the root node at the top, then you must compute the width of the full tree and place that node in the middle of the full width. More accurately, you would have to compute the width of the left and right subtrees and use that to position the root in order to show balanced and unbalanced trees accurately.

On the other hand, if you place the root at the left side of an output line and show the level of nodes as indentation from the leftmost column, it’s easy to print the tree on a terminal. Doing so essentially rotates the tree 90° to the left. Each node of the tree appears on its own line of the output. That allows you to forget about determining the width of subtrees and write a simple recursive method, as shown in [Listing 8-11](#).

Listing 8-11 *Methods to Print Trees with One Node per Line*

```
class BinarySearchTree(object):    # A binary search tree class
...
    def print(self,                # Print the tree sideways with 1 node
               indentBy=4):       # on each line and indenting each level
        self.__pTree(self.__root, # by some blanks. Start at root node
                      "ROOT:  ",   # with no indent
                      indentBy)
```

```

def __pTree(self,          # Recursively print a subtree, sideways
            node,          # with the root node left justified
            nodeType,      # nodeType shows the relation to its
            indent,        # parent and the indent shows its level
            indentBy=4):   # Increase indent level for subtrees
    if node:               # Only print if there is a node
        self.__pTree(node.rightChild, "RIGHT: ", # Print the right
                      indent + " " * indentBy, indentBy) # subtree
        print(indent + nodeType, node) # Print this node
        self.__pTree(node.leftChild, "LEFT: ", # Print the left
                      indent + " " * indentBy, indentBy) # subtree

```

The public `print()` method calls the private `__pTree()` method to recursively print the nodes starting at the root node. It takes a parameter, `indentBy`, to control how many spaces are used to indent each level of the tree. It labels the nodes to show their relationship with their parent (if it wasn't already clear from their indentation and relative positions). The recursive method implementation starts by checking the base case, an empty `node`, in which case nothing needs to be printed. For every other node, it first recursively prints the right subtree because that is the top of the printed version. It adds spaces to the indent so that subtree is printed further to the right. Then it prints the current node prefixed with its indentation and `nodeType` label. Lastly, it prints the left subtree recursively with the extended indentation. This produces an output such as that shown in [Figure 8-25](#). The nodes are printed as `{key, data}` pairs and the figure example has no data stored with it.

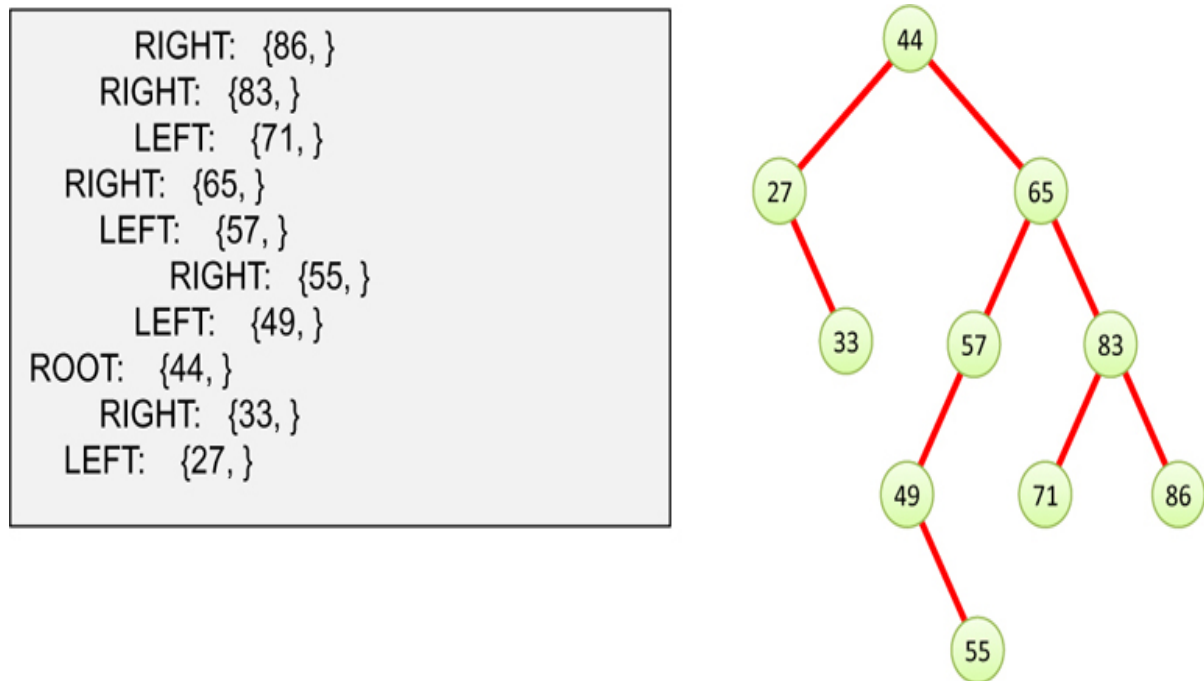


Figure 8-25 Tree printed with indentation for node depth

In printing the tree like this, you use a different traversal order from the three standard ones. The print order uses a *reverse* in-order traversal of the tree.

Duplicate Keys

As in other data structures, the problem of duplicate keys must be addressed. In the code shown for `insert()` and in the visualization tool, a node with a duplicate key will not be inserted. The visualization tool shows the data for the node being updated by moving a new colored circle to fill the node.

To allow for duplicate keys, you must make several choices. The duplicates go in the right subtree based on the fundamental binary search tree rule. They form a chain of nodes with only right child links, as shown in [Figure 8-26](#). One of the design choices is where to put any left child link. It should go only at the first or last duplicate in the chain so that the algorithms know where to find it. The figure illustrates the two choices. New duplicate keys should be inserted at the opposite end of the chain.

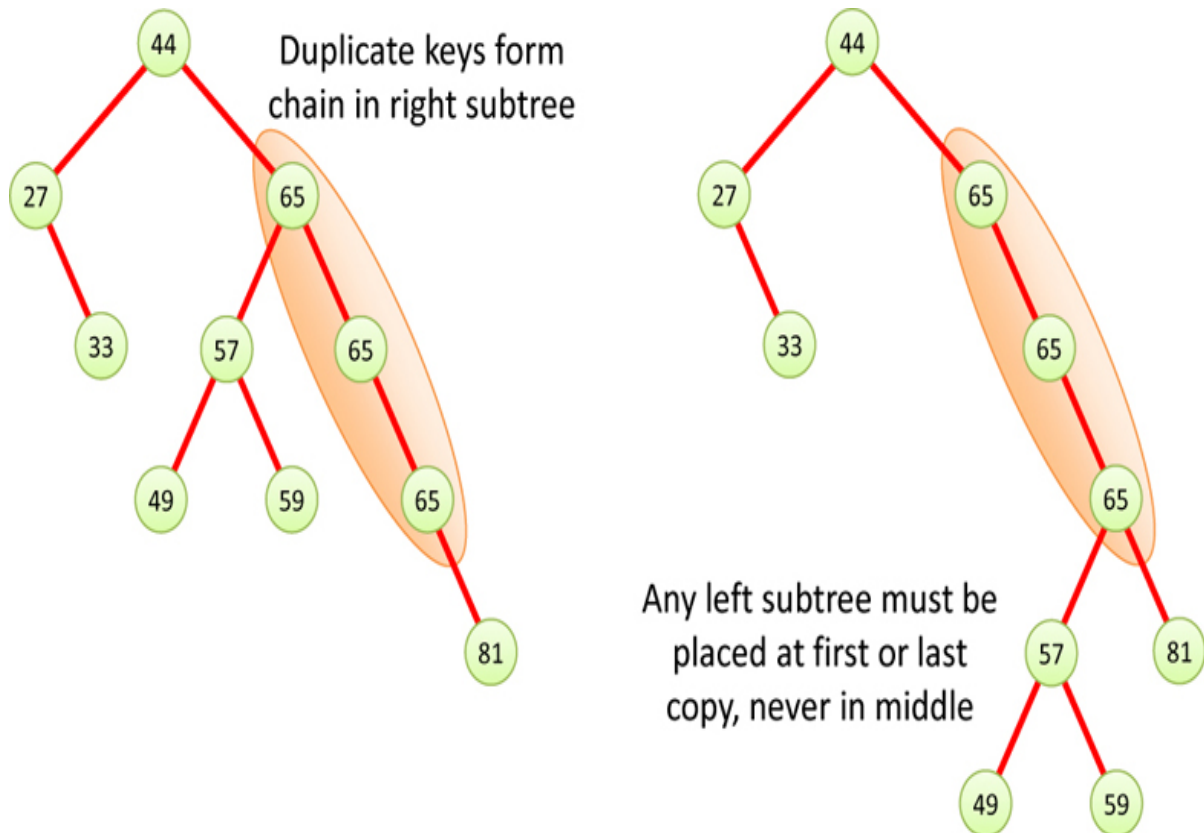


Figure 8-26 *Duplicate keys in binary search trees*

Another choice is what to return from the `__find()` and `search()` methods for a key that has duplicates. Should it return the first or the last? The choice should also be consistent with what node is deleted and returned by the `delete()` method. If they are inserted at the first and removed from the first, then `delete()` will act like a mini stack for the duplicate nodes.

The delete operation is complicated by the fact that different data values could be stored at each of the duplicate nodes. The caller may need to delete a node with specific data, rather than just any node with the duplicate key. Whichever scheme is selected, the deletion routine will need to ensure that the left subtree, if any, remains attached to the appropriate place.

With any kind of duplicate keys, balancing the tree becomes difficult or impossible. The chains of duplicates add extra levels that cannot be rearranged to help with balance. That means the efficiency of finding an item moves away from best case of $O(\log N)$ toward $O(N)$.

As you can see, allowing duplicate keys is not a simple enhancement to the data structure. In other data structures, duplicate keys present challenges, but

not all of them are as tricky as the binary search tree.

The BinarySearchTreeTester.py Program

It's always a good idea to test the functioning of a code module by writing tests that exercise each operation. Writing a comprehensive set of tests is an art in itself. Another useful strategy is to write an interactive test program that allows you to try a series of operations in different orders and with different arguments. To test all the `BinarySearchTree` class methods shown, you can use a program like `BinarySearchTreeTester.py` shown in [Listing 8-12](#).

Listing 8-12 *The BinarySearchTreeTester.py Program*

```
# Test the BinarySearchTree class interactively
from BinarySearchTree import *

theTree = BinarySearchTree() # Start with an empty tree

theTree.insert("Don", "1974 1") # Insert some data
theTree.insert("Herb", "1975 2")
theTree.insert("Ken", "1979 1")
theTree.insert("Ivan", "1988 1")
theTree.insert("Raj", "1994 1")
theTree.insert("Amir", "1996 1")
theTree.insert("Adi", "2002 3")
theTree.insert("Ron", "2002 3")
theTree.insert("Fran", "2006 1")
theTree.insert("Vint", "2006 2")
theTree.insert("Tim", "2016 1")

def print_commands(names): # Print a list of possible commands
    print('The possible commands are', names)

def clearTree(): # Remove all the nodes in the tree
    while not theTree.isEmpty():
        data, key = theTree.root()
        theTree.delete(key)

def traverseTree(traverseType="in"): # Traverse & print all nodes
    for key, data in theTree.traverse(traverseType):
        print('{', str(key), ', ', str(data), '}', end=' ')
    print()
```

```

commands = [ # Command names, functions, and their parameters
    ['print', theTree.print, []],
    ['insert', theTree.insert, ('key', 'data')],
    ['delete', theTree.delete, ('key', )],
    ['search', theTree.search, ('key', )],
    ['traverse', traverseTree, ('type', )],
    ['clear', clearTree, []],
    ['help', print_commands, []],
    ['?', print_commands, []],
    ['quit', None, []],
]

# Collect all the command names in a list
command_names = "\, ".join(c[0] for c in commands)
for i in range(len(commands)): # Put command names in argument list
    if commands[i][1] == print_commands: # of print_commands
        commands[i][2] = [command_names]
# Create a dictionary mapping first character of command name to
# command specification (name, function, parameters/args)
command_dict = dict((c[0][0], c) for c in commands)

# Print information for interactive loop
theTree.print()
print_commands(command_names)
ans = ' '

# Loop to get a command from the user and execute it
while ans[0] != 'q':
    print('The tree has', theTree.nodes(), 'nodes across',
          theTree.levels(), 'levels')
    ans = input("Enter first letter of command: ").lower()
    if len(ans) == 0:
        ans = ' '
    if ans[0] in command_dict:
        name, function, parameters = command_dict[ans[0]]
        if function is not None:
            print(name)
            if isinstance(parameters, list):
                arguments = parameters
            else:
                arguments = []
            for param in parameters:
                arg = input("Enter " + param + " for " + name + " " +
                           "command: ")
                arguments.append(arg)
            try:
                result = function(*arguments)

```

```
        print('Result:', result)
    except Exception as e:
        print('Exception occurred')
        print(e)
else:
    print("Invalid command: '", ans, "'")
```

This program allows users to enter commands by typing them in a terminal interface. It first imports the `BinarySearchTree` module and creates an empty tree with it. Then it puts some data to it, using `insert()` to associate names with some strings. The names are the keys used to place the nodes within the tree.

The tester defines several utility functions to print all the possible commands, clear all the nodes from the tree, and traverse the tree to print each node. These functions handle commands in the command loop below.

The next part of the tester program defines a list of commands. For each one, it has a name, a function to execute the command, and a list or tuple of arguments or parameters. This is more advanced Python code than we've shown so far, so it might look a little strange. The names are what the user will type (or at least their first letter), and the functions are either methods of the tree or the utility functions defined in the tester. The arguments and parameters will be processed after the user chooses a command.

To provide a little command-line help, the tester concatenates the list of command names into a string, separating them with commas. This operation is accomplished with the `join()` method of strings. The text to place between each command name is the string (a comma and a space), and the argument to `join()` is the list of names. The program uses a list comprehension to iterate through the command specifications in `commands` and pull out the first element, which is the command name: `", ".join(c[0] for c in commands)`. The result is stored in the `command_names` variable.

Then the concatenated string of command names needs to get inserted in the argument list for the `print_commands` function. That's done in the `for` loop. Two entries have the `print_commands` function: the `help` and `?` commands.

The last bit of preparation for the command loop creates a dictionary, `command_dict`, that maps the first character of each command to the command specification. You haven't used this Python data structure yet. In [Chapter 11](#), "[Hash Tables](#)," you see how they work, so if you're not familiar with them,

think of them as an **associative array**—an array indexed by a string instead of integer. You can assign values in the array and then look them up quickly. In the tester program, evaluating `command_dict['p']` would return the specification for the print command, namely `['print', theTree.print, []]`. Those specifications get stored in the dictionary using the compact (but cryptic) comprehension: `dict((c[0][0], c) for c in commands)`.

The rest of the tester implements the command loop. It first prints the tree on the terminal, followed by the list of commands. The `ans` variable holds the input typed by the user. It gets initialized to a space so that the command loop starts and prompts for a new command.

The command loop continues until the user invokes the quit command, which starts with `q`. Inside the loop body, the number of nodes and levels in the tree is printed, and then the user is asked for a command. The string that is returned by `input()` is converted to lowercase to simplify the command lookup. If the user just pressed Return, there would be no first character in the string, so you would fill in a `?` to make the default response be to print all the command names again.

In the next statement—if `ans[0] in command_dict`:—the tester checks whether the first character in the user's response is one of the known commands. If the character is recognized, it extracts the `name`, `function`, and `parameters` from the specification stored in the `command_dict`. If there's a function to execute, then it will be processed. If not, then the user asked to quit, and the `while` loop will exit. When the first character of the user's response does not match a command, an error message is printed, and the loop prompts for a new command.

After the command specification is found, it either needs to prompt the user for the arguments to use when calling the `function` or get them from the specification. This choice is based on whether the parameters were specified as Python tuple or list. If it's a tuple, the elements of the tuple are the names of the `parameters`. If it's a list, then the list contains the `arguments` of the function. For tuples, the user is prompted to enter each argument by name, and the answers are stored in the `arguments` list. After the `arguments` are determined, the command loop tries calling the function with the `arguments` list using `result = function(*arguments)`. The asterisk (`*`) before the `arguments` is not a multiplication operator. It means that the `arguments` list should be used as the list of positional arguments for the function. If the function raises any

exceptions, they are caught and displayed. Otherwise, the result of the function is printed before looping to get another command.

Try using the tester to run the four main operations: search, insert, traverse, and delete. For the deletion, try deleting nodes with 0, 1, and 2 child nodes to see the effect. When you delete a node with 2 children, predict which successor node will replace the deleted node and see whether you're right.

The Huffman Code

You shouldn't get the idea that binary trees are always search trees. Many binary trees are used in other ways. [Figure 8-16](#) shows an example where a binary tree represents an algebraic expression. We now discuss an algorithm that uses a binary tree in a surprising way to compress data. It's called the Huffman code, after David Huffman who discovered it in 1952. Data compression is important in many situations. An example is sending data over the Internet or via digital broadcasts, where it's important to send the information in its shortest form. Compressing the data means more data can be sent in the same time under the bandwidth limits.

Character Codes

Each character in an uncompressed text file is represented in the computer by one to four bytes, depending on the way characters are encoded. For the venerable ASCII code, only one byte is used, but that limits the range of characters that can be expressed to fewer than 128. To account for all the world's languages plus other symbols like emojis 😊, the various Unicode standards use up to four bytes per character. For this discussion, we assume that only the ASCII characters are needed, and each character takes one byte (or eight bits). [Table 8-2](#) shows how some characters are represented in binary using the ASCII code.

Table 8-2 *Some ASCII Codes*

Character	Decimal	Binary
@	64	01000000
A	65	01000001
B	66	01000010
...	...	...
Y	89	01011001
Z	90	01011010
...	...	...
a	97	01100001
b	98	01100010

There are several approaches to compressing data. For text, the most common approach is to reduce the number of bits that represent the most-used characters. As a consequence, each character takes a variable number of bits in the “stream” of bits that represents the full text.

In English, *E* and *T* are very common letters, when examining prose and other person-to-person communication and ignoring things like spaces and punctuation. If you choose a scheme that uses only a few bits to write *E*, *T*, and other common letters, it should be more compact than if you use the same number of bits for every letter. On the other end of the spectrum, *Q* and *Z* seldom appear, so using a large number of bits occasionally for those letters is not so bad.

Suppose you use just two bits for *E*—say 01. You can’t encode every letter of the English alphabet in two bits because there are only four 2-bit combinations: 00, 01, 10, and 11. Can you use these four combinations for the four most-used characters? Well, if you did, and you still wanted to have some encoding for the lesser-used characters, you would have trouble. The algorithm that interprets the bits would have to somehow guess whether a pair of bits is a single character or part of some longer character code.

One of the key ideas in encoding is that we must set aside some of the code values as indicators that a longer bit string follows to encode a lesser-used character. The algorithm needs a way to look at a bit string of a particular length and determine if that is the full code for one of the characters or just a prefix for a longer code value. You must be careful that no character is

represented by the same bit combination that appears at the beginning of a longer code used for some other character. For example, if *E* is 01, and *Z* is 01011000, then an algorithm decoding 01011000 wouldn't know whether the initial 01 represented an *E* or the beginning of a *Z*. This leads to a rule: *No code can be the prefix of any other code.*

Consider also that in some messages, *E* might not be the most-used character. If the text is a program source file, for example, punctuation characters such as the colon (:), semicolon (;), and underscore (_) might appear more often than *E* does. Here's a solution to that problem: for each message, you make up a new code tailored to that particular message. Suppose you want to send the message SPAM SPAM SPAM EGG + SPAM. The letter *S* appears a lot, and so does the space character. You might want to make up a table showing how many times each letter appears. This is called a frequency table, as shown in [Table 8-3](#).

Table 8-3 *Frequency Table for the SPAM Message*

Character	Count		Character	Count
A	4		P	4
E	1		S	4
G	2		Space	5
M	4		+	1

The characters with the highest counts should be coded with a small number of bits. [Table 8-4](#) shows one way how you might encode the characters in the SPAM message.

Table 8-4 *Huffman Code for the SPAM Message*

Character	Count	Code		Character	Count	Code
A	4	111		P	4	110
E	1	10000		S	4	101
G	2	1001		Space	5	01
M	4	00		+	1	10001

You can use 01 for the space because it is the most frequent. The next most frequent characters are *S*, *P*, *A*, and *M*, each one appearing four times. You use the code 00 for the last one, *M*. The remaining codes can't start with 00 or 01

because that would break the rule that no code can be a prefix of another code. That leaves 10 and 11 to use as prefixes for the other characters.

What about 3-bit code combinations? There are eight possibilities: 000, 001, 010, 011, 100, 101, 110, and 111, but you already know you can't use anything starting with 00 or 01. That eliminates four possibilities. You can assign some of those 3-bit codes to the next most frequent characters, *S* as 101, *P* as 110, and *A* as 111. That leaves the prefix 100 to use for the remaining characters. You use a 4-bit code, 1001, for the next most frequent character, *G*, which appears twice. There are two characters that appear only once, *E* and *+*. They are encoded with 5-bit codes, 10000 and 10001.

Thus, the entire message is coded as

```
101 110 111 00 01 101 110 111 00 01 101 110 111 00 01 10000 1001 1001
01 10001 01 101 110 111 00
```

For legibility, we show this message broken into the codes for individual characters. Of course, all the bits would run together because there is no space character in a binary message, only 0s and 1s. That makes it more challenging to find which bits correspond to a character. The main point, however, is that the 25 characters in the input message, which would typically be stored in 200 bits in memory (8×25), require only 72 bits in the Huffman coding.

Decoding with the Huffman Tree

We show later how to create Huffman codes. First, let's examine the somewhat easier process of decoding. Suppose you received the string of bits shown in the preceding section. How would you transform it back into characters? You could use a kind of binary tree called a **Huffman tree**. [Figure 8-27](#) shows the Huffman tree for the SPAM message just discussed.

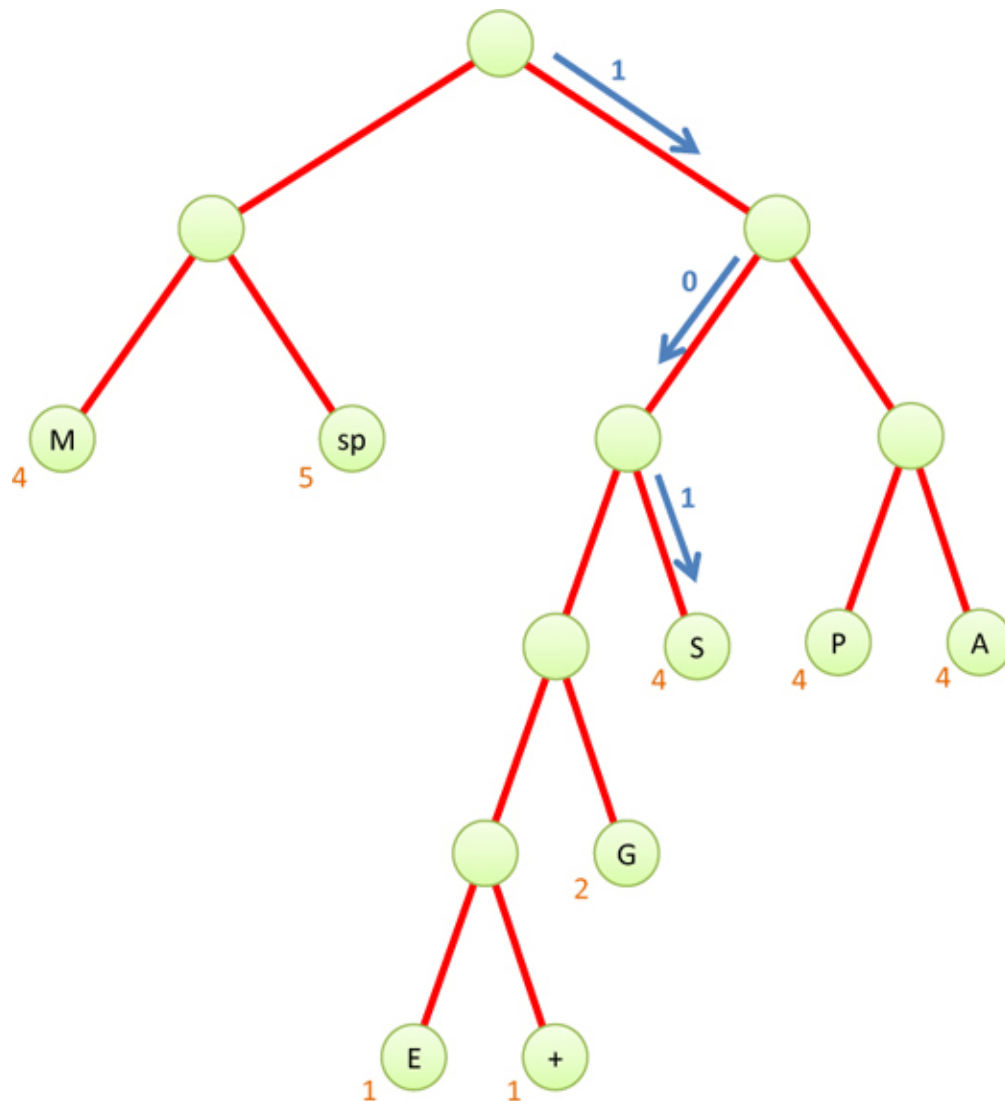


Figure 8-27 *Huffman tree for the SPAM message*

The characters in the message appear in the tree as leaf nodes. The higher their frequency in the message, the higher up they appear in the tree. The number outside each leaf node is its frequency. That puts the space character (sp) at the second level, and the *S*, *P*, *A*, and *M* characters at the second or third level. The least frequent, *E* and +, are on the lowest level, 5.

How do you use this tree to decode the message? You start by looking at the first bit of the message and set a pointer to the root node of the tree. If you see a 0 bit, you move the pointer to the left child of the node, and if you see a 1 bit, you move it right. If the identified node does not have an associated character, then you advance to the next bit in the message. Try it with the code for *S*,

which is 101. You go right, left, then right again, and voila, you find yourself on the *S* node. This is shown by the blue arrows in [Figure 8-27](#).

You can do the same with the other characters. After you've arrived at a leaf node, you can add its character to the decoded string and move the pointer back to the root node. If you have the patience, you can decode the entire bit string this way.

Creating the Huffman Tree

You've seen how to use a Huffman tree for decoding, but how do you create this tree? There are many ways to handle this problem. You need a Huffman tree object, and that is somewhat like the `BinarySearchTree` described previously in that it has nodes that have up to two child nodes. It's quite different, however, because routines that are specific to search trees, like `find()`, `insert()`, and `delete()`, are not relevant. The constraint that a node's key be larger than any key of its left child and equal to or less than any key of its right child doesn't apply to a Huffman tree. Let's call the new class `HuffmanTree`, and like the search tree, store a key and a value at each node.

Here is the algorithm for constructing a Huffman tree from a message string:

Preparation

1. Count how many times each character appears in the message string.
2. Make a `HuffmanTree` object for each character used in the message. For the SPAM message example, that would be eight trees. Each tree has a single node whose key is a character and whose value is that character's frequency in the message. Those values can be found in [Table 8-3](#) or [Table 8-4](#) for the SPAM message.
3. Insert these trees in a priority queue (as described in [Chapter 4](#)). They are ordered by the frequency (stored as the value of each root node) and the number of levels in the tree. The tree with the smallest frequency has the highest priority. Among trees with equal frequency, the one with more levels is the highest priority. In other words, when you remove a tree from the priority queue, it's always the one with the deepest tree of the least-used character. (Breaking ties using the tree depth, improves the balance of the final Huffman tree.)

That completes the preparation, as shown in Step 0 of [Figure 8-28](#).

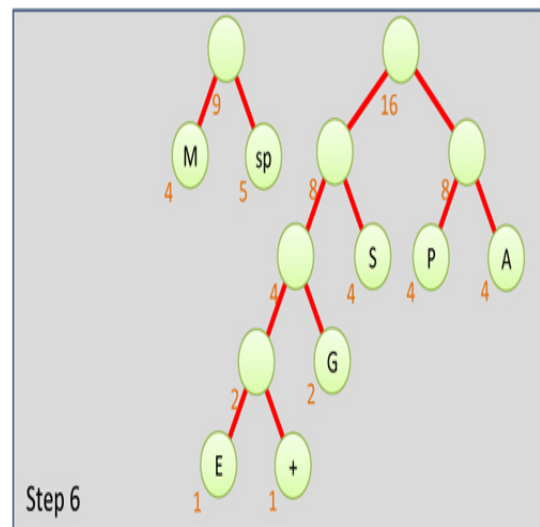
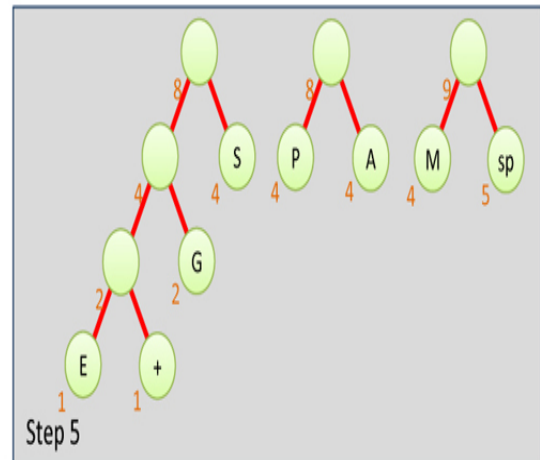
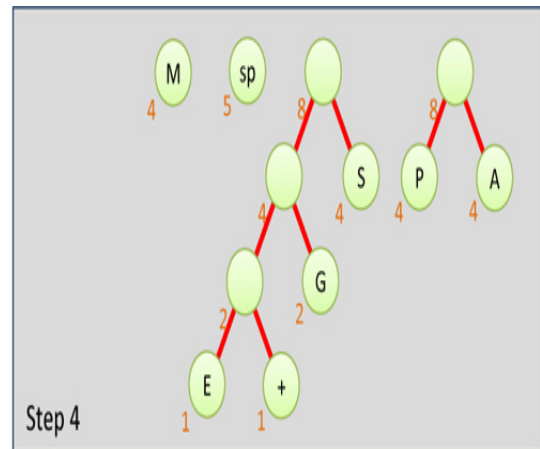
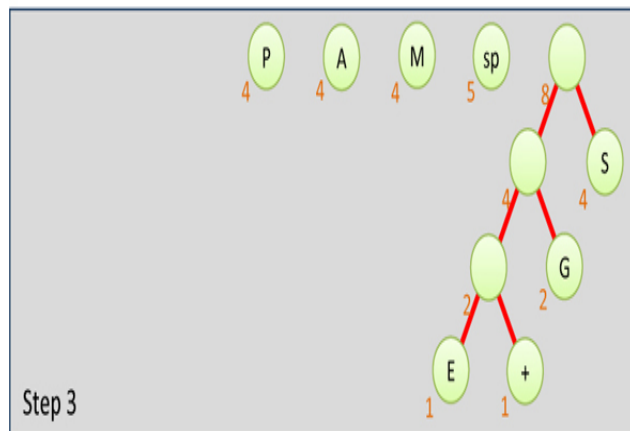
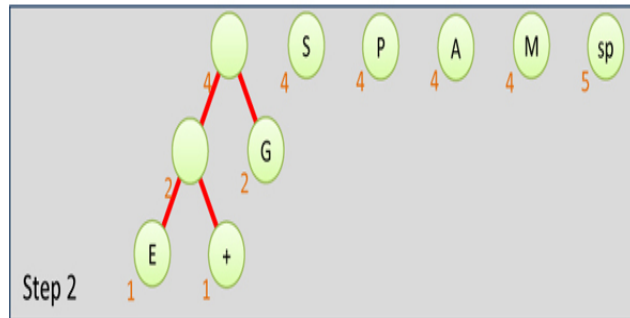
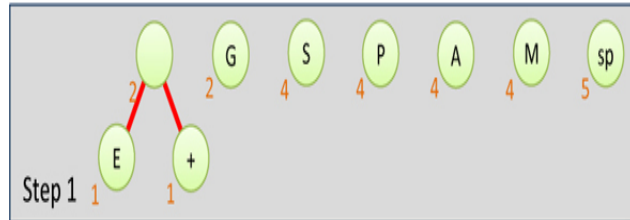


Figure 8-28 Growing the Huffman tree, first six steps

Then do the following:

Tree consolidation

1. Remove two trees from the priority queue and make them into children of a new node. The new node has a frequency value that is the sum of the children's frequencies; its character key can be left blank.
2. Insert this new, deeper tree back into the priority queue.
3. Keep repeating steps 1 and 2. The trees will get larger and larger, and there will be fewer and fewer of them. When there is only one tree left in the priority queue, it is the Huffman tree and you're done.

[Figure 8-28](#) and [Figure 8-29](#) show how the Huffman tree is constructed for the SPAM message.

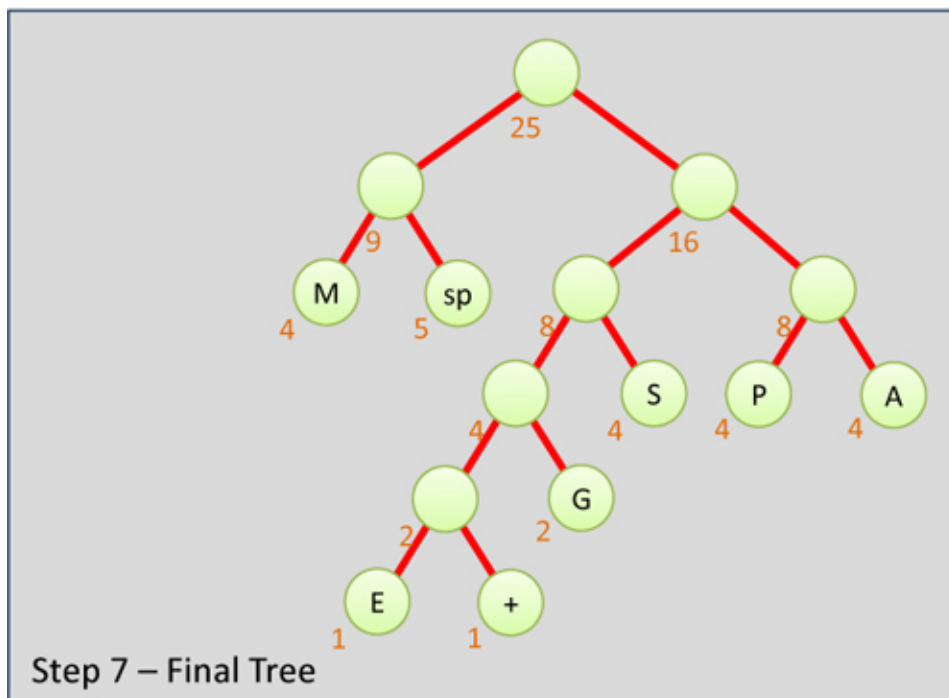
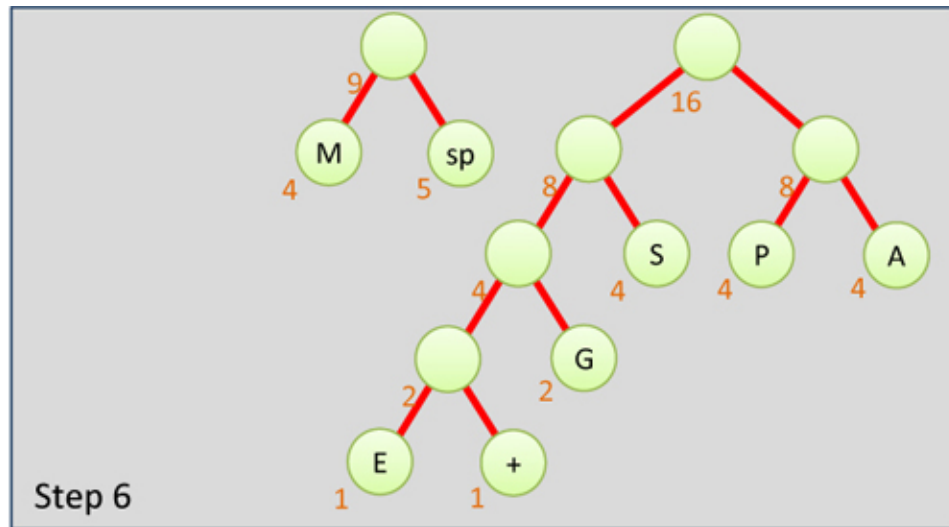


Figure 8-29 *Growing the Huffman tree, final step*

Coding the Message

Now that you have the Huffman tree, how do you encode a message? You start by creating a code table, which lists the Huffman code alongside each character. To simplify the discussion, we continue to assume that only ASCII characters are possible, so we need a table with 128 cells. The index of each

cell would be the numerical value of the ASCII character: 65 for A, 66 for B, and so on. The contents of the cell would be the Huffman code for the corresponding character. Initially, you could fill in some special value for indicating “no code” like `None` or an empty string in Python to check for errors where you failed to make a code for some character.

Such a code table makes it easy to generate the coded message: for each character in the original message, you use its code as an index into the code table. You then repeatedly append the Huffman codes to the end of the coded message until it’s complete.

To fill in the codes in the table, you traverse the Huffman tree, keeping track of the path to each node as it is visited. When you visit a leaf node, you use the key for that node as the index to the table and insert the path as a binary string into the cell’s value. Not every cell contains a code—only those appearing in the message. [Figure 8-30](#) shows how this looks for the SPAM message. The table is abbreviated to show only the significant rows. The path to the leaf node for character *G* is shown as the tree is being traversed.

The full code table can be built by calling a method that starts at the root and then calls itself recursively for each child. Eventually, the paths to all the leaf nodes will be explored, and the code table will be complete.

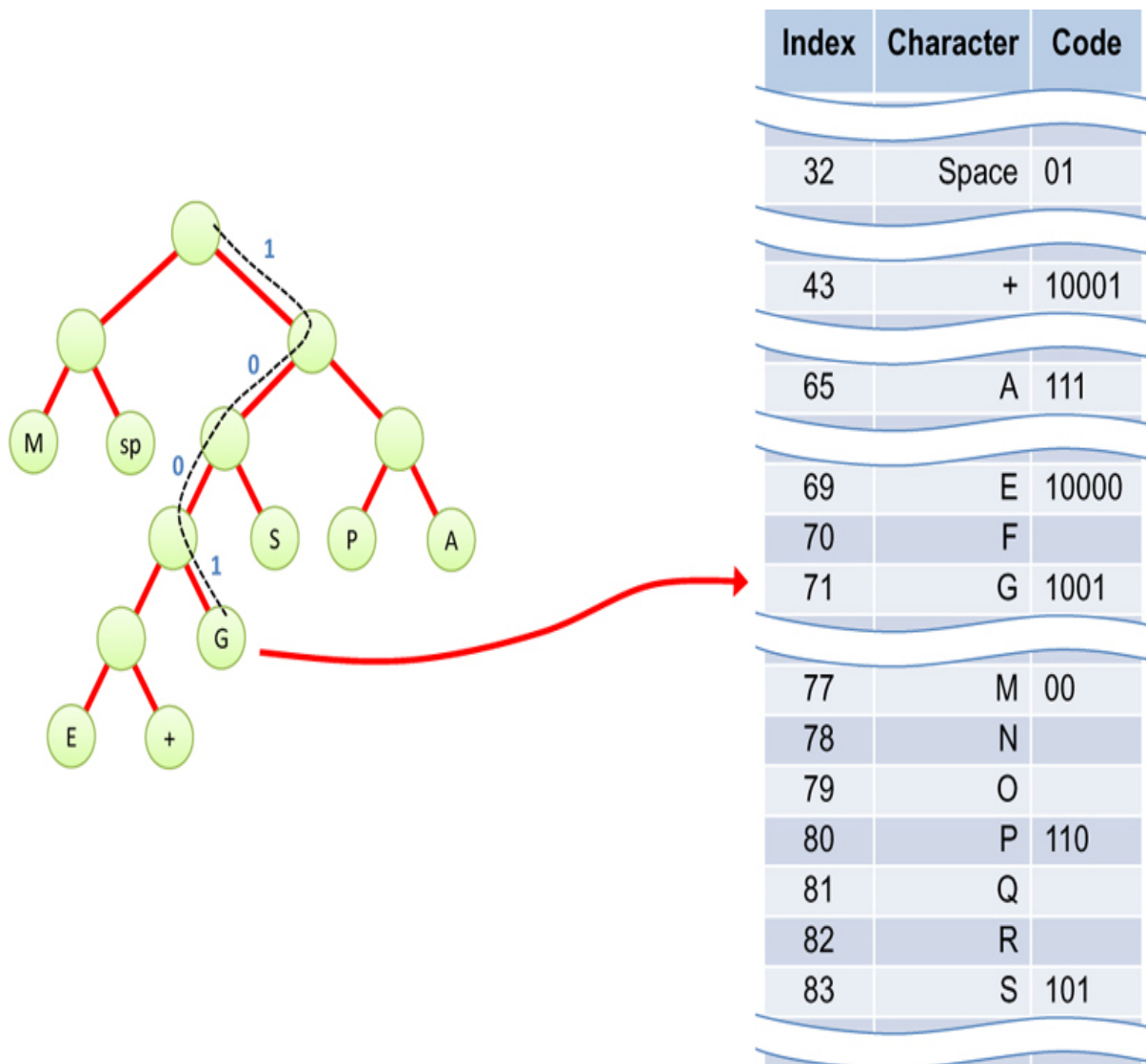


Figure 8-30 *Building the code table*

One more thing to consider: if you receive a binary message that's been compressed with a Huffman code, how do you know what Huffman tree to use for decoding it? The answer is that the Huffman tree must be sent first, before the binary message, in some format that doesn't require knowledge of the message content. Remember that Huffman codes are for *compressing the data*, not encrypting it. Sending a short description of the Huffman tree followed by a compressed version of a long message saves many bits.

Summary

- Trees consist of nodes connected by edges.
- The root is the topmost node in a tree; it has no parent.
- All nodes but the root in a tree have exactly one parent.
- In a binary tree, a node has at most two children.
- Leaf nodes in a tree have no child nodes and exactly one path to the root.
- An unbalanced tree is one whose root has many more left descendants than right descendants, or vice versa.
- Each node of a tree stores some data. The data typically has a key value used to identify it.
- Edges are most commonly represented by references to a node's children; less common are references from a node to its parent.
- Traversing a tree means visiting all its nodes in some predefined order.
- The simplest traversals are pre-order, in-order, and post-order.
- Pre-order and post-order traversals are useful for parsing algebraic expressions.
- *Binary Search Trees*
 - In a binary search tree, all the nodes that are left descendants of node A have key values less than that of A; all the nodes that are A's right descendants have key values greater than (or equal to) that of A.
 - Binary search trees perform searches, insertions, and deletions in $O(\log N)$ time.
 - Searching for a node in a binary search tree involves comparing the goal key to be found with the key value of a node and going to that node's left child if the goal key is less or to the node's right child if the goal key is greater.
 - Insertion involves finding the place to insert the new node and then changing a child field in its new parent to refer to it.
 - An in-order traversal visits nodes in order of ascending keys.

- When a node has no children, you can delete it by clearing the child field in its parent (for example, setting it to `None` in Python).
- When a node has one child, you can delete it by setting the child field in its parent to point to its child.
- When a node has two children, you can delete it by replacing it with its successor and deleting the successor from the subtree.
- You can find the successor to a node A by finding the minimum node in A's right subtree.
- Nodes with duplicate key values require extra coding because typically only one of them (the first) is found in a search, and managing their children complicates insertions and deletions.
- Trees can be represented in the computer's memory as an array, although the reference-based approach is more common and memory efficient.
- A Huffman tree is a binary tree (but not a search tree) used in a data-compression algorithm called Huffman coding.
- In the Huffman code, the characters that appear most frequently are coded with the fewest bits, and those that appear rarely are coded with the most bits.
- The paths in the Huffman tree provide the codes for each of the leaf nodes.
- The level of a leaf node indicates the number of bits used in the code for its key.
- The characters appearing the least frequently in a Huffman coded message are placed in leaf nodes at the deepest levels of the Huffman tree.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. Insertion and deletion in a binary search tree require what Big O time?

2. A binary tree is a search tree if
 - a. every nonleaf node has children whose key values are less than or equal to the parent.
 - b. the key values of every nonleaf node are the sum or concatenation of the keys of its children
 - c. every left child has a key less than its parent and every right child has a key greater than or equal to its parent.
 - d. in the path from the root to every leaf node, the key of each node is greater than or equal to the key of its parent.
3. True or False: If you traverse a tree and print the path to each node as a series of the letters *L* and *R* for whether the path followed the left or right child at each step, there could be some duplicate paths.
4. When compared to storing data in an ordered array, the main benefit of storing it in a binary search tree is
 - a. having the same search time as traversal time in Big O notation.
 - b. not having to copy data when inserting or deleting items.
 - c. being able to search for an item in $O(\log N)$ time.
 - d. having a key that is separate from the value identified by the key.
5. In a complete, balanced binary tree with 20 nodes, and the root considered to be at level 0, how many nodes are there at level 4?
6. A subtree of a binary tree always has
 - a. a root that is a child of the main tree's root.
 - b. a root unconnected to the main tree's root.
 - c. fewer nodes than the main tree.
 - d. a sibling with an equal or larger number of nodes.
7. When implementing trees as objects, the _____ and the _____ are generally separate classes.
8. Finding a node in a binary search tree involves going from node to node, asking
 - a. how big the node's key is in relation to the search key.

- b. how big the node's key is compared to its right or left child's key.
- c. what leaf node you want to reach.
- d. whether the level you are on is above or below the search key.

9. An unbalanced tree is one

- a. in which most of the keys have values greater than the average.
- b. where there are more nodes above the central node than below.
- c. where the leaf nodes appear much more frequently as the left child of their parents than as the right child, or vice versa.
- d. in which the root or some other node has many more left descendants than right descendants, or vice versa.

10. True or False: A hierarchical file system is essentially a binary search tree, although it can be unbalanced.

11. Inserting a node starts with the same steps as _____ a node.

12. Traversing tree data structures

- a. requires multiple methods to handle the different traversal orders.
- b. can be implemented using recursive functions or generators.
- c. is much faster than traversing array data structures.
- d. is a way to make soft deletion of items practical.

13. When a tree is extremely unbalanced, it begins to behave like the _____ data structure.

14. Suppose a node A has a successor node S in a binary search tree with no duplicate keys. Then S must have a key that is larger than _____ but smaller than or equal to _____.

15. Deleting nodes in a binary search tree is complex because

- a. copying subtrees below the successor requires another traversal.
- b. finding the successor is difficult to do, especially when the tree is unbalanced.
- c. the tree can split into multiple trees, a forest, if it's not done properly.
- d. the operation is very different for the different number of child nodes of the node to be deleted, 0, 1, or 2.

16. In a binary tree used to represent a mathematical expression,
 - a. both children of an operator node must be operands.
 - b. following a post-order traversal, parentheses must be added.
 - c. following a pre-order traversal, parentheses must be added.
 - d. in pre-order traversal, a node is visited before either of its children.
17. When a tree is represented by an array, the right child of a node at index n has an index of _____.
18. True or False: Deleting a node with one child from a binary search tree involves finding that node's successor.
19. A Huffman tree is typically used to _____ text data.
20. Which of the following is **not** true about a Huffman tree?
 - a. The most frequently used characters always appear near the top of the tree.
 - b. Normally, decoding a message involves repeatedly following a path from the root to a leaf.
 - c. In coding a character, you typically start at a leaf and work upward.
 - d. The tree can be generated by removal and insertion operations on a priority queue of small trees.

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

- 8-A Use the Binary Search Tree Visualization tool to create 20 random trees using 20 as the requested number of items. What percentage would you say are seriously unbalanced?
- 8-B Use the `BinarySearchTreeTester.py` program shown in [Listing 8-12](#) and provided with the code examples from the publisher's website to do the following experiments:
 - a. Delete a node that has no children.
 - b. Delete a node that has 1 child node.

- c. Delete a node that has 2 child nodes.
 - d. Pick a key for a new node to insert. Determine where you think it will be inserted in the tree, and then insert it with the program. Is it easy to determine where it will go?
 - e. Repeat the previous step with another key but try to put it in the other child branch. For example, if your first node was inserted as the left child, try to put one as the right child or in the right subtree.
- 8-C The `BinarySearchTreeTester.py` program shown in [Listing 8-12](#) prints an initial tree of 11 nodes across 7 levels, based on the insertion order of the items. A fully balanced version of the tree would have the same nodes stored on 4 levels. Use the program to clear the tree, and then determine what order to insert the same keys to make a balanced tree. Try your ordering and see whether the tree comes out balanced. If not, try another ordering. Can you describe in a few sentences the insertion ordering that will always create a balanced binary search tree from a particular set of keys?
- 8-D Use the Binary Search Tree Visualization tool to delete a node in every possible situation.

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

- 8.1 Alter the `BinarySearchTree` class described in this chapter to allow nodes with duplicate keys. Three methods are affected: `__find()`, `insert()`, and `delete()`. Choose to insert new left children at the shallowest level among equal keys, as shown on the left side of [Figure 8-26](#), and always find and delete the deepest among equal keys. More specifically, the `__find()` and `search()` methods should return the deepest among equal keys that it encounters but should allow an optional parameter to specify finding the shallowest. The `insert()` method must handle the case when the item to be inserted duplicates an existing node, by inserting a new node with an empty left child below the deepest duplicate key. The `delete()` method must delete the deepest

node among duplicate keys, thus providing a LIFO or stack-like behavior among duplicate keys. Think carefully about the deletion cases and whether the choice of successor nodes changes. Demonstrate how your implementation works on a tree inserting several duplicate keys associated with different values. Then delete those keys and show their values to make it clear that the last duplicate inserted is the first duplicate deleted.

- 8.2 Write a program that takes a string containing a postfix expression and builds a binary tree to represent the algebraic expression like that shown in [Figure 8-16](#). You need a `BinaryTree` class, like that of

`BinarySearchTree`, but without any keys or ordering of the nodes.

Instead of `find()`, `insert()`, and `delete()` methods, you need the ability to make single node `BinaryTrees` containing a single operand and a method to combine two binary trees to make a third with an operator as the root node. The syntax of the operators and operands is the same as what was used in the `PostfixTranslate.py` module from [Chapter 4](#). You can use the `nextToken()` function in that module to parse the input string into operator and operand tokens. You don't need the parentheses as delimiters because postfix expressions don't use them. Verify that the input expression produces a single algebraic expression and raise an exception if it does not. For valid algebraic binary trees, use pre-, in-, and post-order traversals of the tree to translate the input into the output forms. Include parentheses for the in-order traversal to make the operator precedence clear in the output translation. Run your program on at least the following expressions:

- a. 91 95 + 15 + 19 + 4 *
- b. B B * A C 4 * * -
- c. 42
- d. A 57 # this should produce an exception
- e. + / # this should produce an exception

- 8.3 Write a program to implement Huffman coding and decoding. It should do the following:

- Accept a text message (string).
- Create a Huffman tree for this message.

- Create a code table.
- Encode the text message into binary.
- Decode the binary message back to text.
- Show the number of bits in the binary message and the number of characters in the input message.

If the message is short, the program should be able to display the Huffman tree after creating it. You can use Python `string` variables to store binary messages as arrangements of the characters 1 and 0. Don't worry about doing actual bit manipulation using `bytearray` unless you really want to. The easiest way to create the code table in Python is to use the dictionary (`dict`) data type. If that is unfamiliar, it's essentially an array that can be indexed by a string or a single character. It's used in the `BinarySearchTreeTester.py` module shown in [Listing 8-12](#) to map command letters to command records. If you choose to use an integer indexed array, you can use Python's `ord()` function to convert a character to an integer but be aware that you will need a large array if you allow arbitrary Unicode characters such as emojis (😊) in the message.

8.4 Measuring tree balance can be tricky. You can apply two simple measures: node balance and level (or height) balance. As mentioned previously, balanced trees have an approximately equal number of nodes in their left and right subtrees. Similarly, the left and right subtrees must have an approximately equal number of levels (or height). Extend the `BinarySearchTree` class by writing the following methods:

- `nodeBalance()`—Computes the number of nodes in the right subtree minus the number of nodes in the left subtree
- `levelBalance()`—Computes the number of levels in the right subtree minus the number of levels in the left subtree
- `unbalancedNodes(by=1)`—Returns a list of node keys where the absolute value of either of the balance metrics exceeds the `by` threshold, which defaults to 1

These three methods all require (recursive) helper methods that traverse subtrees rooted at nodes inside the tree. In a balanced tree, the list of unbalanced nodes would be empty. Try your measures by inserting the

following four lists of keys into an empty `BinarySearchTree` (in order, left to right), printing the resulting 15-node tree, printing the node and level balance of the resulting root node, and then printing the list of unbalanced keys with `by=1` and `by=2`.

```
[7, 6, 5, 4, 3, 2, 1, 8, 12, 10, 9, 11, 14, 13, 15],
[8, 4, 5, 6, 7, 3, 2, 1, 12, 10, 9, 11, 14, 13, 15],
[8, 4, 2, 3, 1, 6, 5, 7, 12, 10, 9, 11, 14, 13, 15],
[8, 4, 2, 3, 1, 6, 5, 7, 12, 10, 9, 11, 14, 13, 8.5]
```

8.5 Every binary tree can be represented as an array, as described in the section titled “[Trees Represented as Arrays](#).” The reverse of representing an array as a tree, however, works only for some arrays. The missing nodes of the tree are represented in the array cells as some predefined value—such as `None`—that cannot be a value stored at a tree node. If the root node is missing in the array, then the corresponding tree cannot be built. Write a function that takes an array as input and tries to make a binary tree from its contents. Every cell that is not `None` is a value to store at a tree node. When you come across a node without a parent node (other than the root node), the function should raise an exception indicating that the tree cannot be built. Note that the result won’t necessarily be a binary search tree, just a binary tree. Hint: It’s easier to work from the leaf nodes to the root, building nodes for each cell that is not `None` and storing the resulting node back in the same cell of the input array for retrieval when it is used as a subtree of a node on another level. Print the result of running the function on the following arrays where `n = None`. The values in the array can be stored as either the key or the value of the node because the tree won’t be interpreted as a binary search tree.

```
[],
[n, n, n],
[55, 12, 71],
[55, 12, n, 4],
[55, 12, n, 4, n, n, n, n, 8, n, n, n, n, n, n, n, 6, n],
[55, 12, n, n, n, n, 4, n, 8, n, n, n, n, n, n, n, 6, n]
```

9. 2-3-4 Trees and External Storage

In This Chapter

- [Introduction to 2-3-4 Trees](#)
- [The Tree234 Visualization Tool](#)
- [Python Code for a 2-3-4 Tree](#)
- [Efficiency of 2-3-4 Trees](#)
- [2-3 Trees](#)
- [External Storage](#)

In a binary tree, each node has one data item and can have up to two children. If you allow more data items and children per node, the result is a *multiway tree*. A 2-3-4 tree, to which we devote the first part of this chapter, is a multiway tree that can have up to four children and three data items per node.

These 2-3-4 trees are interesting for several reasons. First, they're balanced trees designed to avoid the problems that come from having too many nodes along some of the paths in the tree. Second, there's an interesting way to convert them into binary trees that we describe in a later chapter. Third, and most important, they serve as an easy-to-understand introduction to B-trees.

A B-tree is another kind of multiway tree that's particularly useful for organizing data in external storage. In this case, *external* means external to main memory, usually a disk drive. A node in a B-tree can have dozens or hundreds of children. We discuss external storage and B-trees at the end of this chapter.

Introduction to 2-3-4 Trees

In this section we look at the characteristics of 2-3-4 trees. Later we show how a Visualization tool models a 2-3-4 tree and how to program a 2-3-4 tree in Python. [Figure 9-1](#) shows a small 2-3-4 tree. Each lozenge-shaped node can hold one, two, or three data items, making it a kind of **multiway tree**.

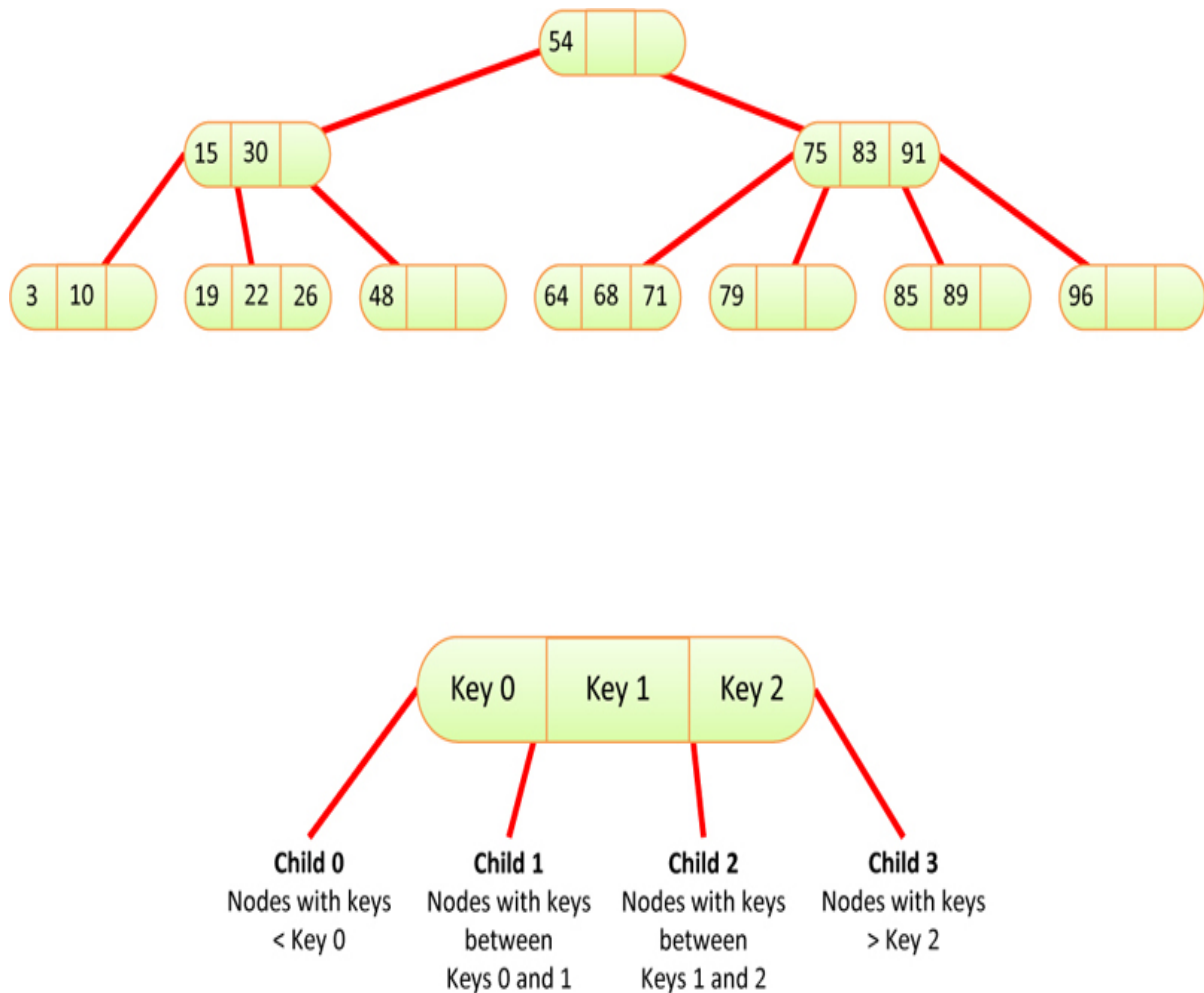


Figure 9-1 *A 2-3-4 tree*

Here, the top three nodes have children, and the seven nodes on the bottom row are all leaf nodes, which have no children, by definition. In a 2-3-4 tree, all the leaf nodes are always on the same level. Having all the leaves at the same level ensures the balance.

What's in a Name?

The 2, 3, and 4 in the name **2-3-4 tree** refer to how many links to child nodes can potentially be referenced by a given node. For nonleaf nodes, three

arrangements are possible:

- A node with one data item always has two children.
- A node with two data items always has three children.
- A node with three data items always has four children.

In short, nonleaf nodes must always have one more child than they have data items. Or, to put it symbolically, if the number of child links is L and the number of data items is D , then

$$D = 1 \quad \text{or} \quad D = 2 \quad \text{or} \quad D = 3$$

$$L = 0 \quad \text{or} \quad L = D + 1$$

This critical relationship determines the structure of 2-3-4 trees. A leaf node has no children, but it can nevertheless contain one, two, or three data items. Empty nodes are not allowed. [Figure 9-1](#) shows an example of each kind of node. There's a node with one data item and two children at the root. Below that, there is a node with two data items and three children on the left, and a node with three data items and four children on the right. The rest are leaf nodes with one, two, or three data items.

Because a 2-3-4 tree can have nodes with up to four children, it's called a *multiway tree of order 4*.

You may wonder why a 2-3-4 tree isn't called a 1-2-3-4 tree. Can't a node have only one child, as some nodes do in binary trees? A binary tree—described in [Chapter 8](#), “[Binary Trees](#)”—might be thought of as a multiway tree of order 2 because each node can have up to two children. There are two important differences, however, between binary trees and 2-3-4 trees. The obvious one is the maximum number of children—four in the case of 2-3-4 trees. In a binary tree, a node can have *up to* two child links. A single link, to its left or to its right child, is also perfectly permissible. The other link has an empty or `None` value.

In a 2-3-4 tree, on the other hand, there is a *minimum number* of two children for nonleaf nodes; internal nodes with a single link are not permitted. A node with one data item must either have two links or be a leaf, in which case it has no links.

2-3-4 Tree Terminology

The nodes in 2-3-4 trees are named for their number of child links. You might be tempted to call them by the number of data items they contain, but that could leave some doubt as to whether they are leaf nodes or not. Instead, you call the root node of [Figure 9-1](#) a *2-node* because it has two children (and, by rule, one data item, 54). The child to the left of the root is a *3-node* because it has three children (and two data items, 15 and 30). The final internal node is a *4-node* with three data items: 75, 83, and 91. The bottom nodes are still called leaf nodes, not 0-nodes.

For convenience, we number the data items in a node from 0 to 2, and the child links from 0 to 3. The data items in a node are arranged in ascending key order, by convention from left to right (lower to higher numbers). So, instead of left, middle, and right, we refer to data (or key) 0, data 1, data 2, child 0, child 1, child 2, and child 3.

2-3-4 Tree Organization

An important aspect of any tree's structure is the relationship of its links to the key values of its data items. In a binary search tree, all children with keys less than the node's key are in a subtree rooted in the node's left child, and all children with keys larger than or equal to the node's key are rooted in the node's right child. In a 2-3-4 tree, the principle is the same because they are always used for searching, but there's more to it:

- All children in the subtree rooted at child 0 have key values less than key 0.
- All children in the subtree rooted at child 1 have key values greater than key 0 but less than key 1.
- All children in the subtree rooted at child 2 have key values greater than key 1 but less than key 2.
- All children in the subtree rooted at child 3 have key values greater than key 2.

The keys are stored in ascending order within a node, usually inside an array. They are named by the indices to the array, not by labels such as “left,”

“middle,” and “right.” This relationship is shown in [Figure 9-2](#). Duplicate values are not usually permitted in 2-3-4 trees, so you don’t need to worry about comparing equal keys.



Figure 9-2 *Keys and children*

Look again at the tree in [Figure 9-1](#). As in all 2-3-4 trees, the leaves are all on the same level (the bottom row). Upper-level nodes are often not full; that is, they may contain only one or two data items instead of three. Also, notice that the tree is balanced. It retains its balance even if you insert a sequence of data in ascending (or descending) order. The 2-3-4 tree’s self-balancing capability results from the way new data items are inserted, as we’ll describe in a moment.

Searching a 2-3-4 Tree

Finding a data item with a particular key is like the search routine in a binary tree. You start at the root and, unless the search key is found there, select the link that leads to the subtree with the appropriate range of values.

For example, to search for the data item with key 89 in the tree in [Figure 9-1](#), you start at the root. You search the root but don’t find 89 among its keys. Because 89 is larger than 54, you go to child 1, which is represented as 75-83-91. (Remember that child 1 is on the right because the numbering of children and links starts at 0 on the left.) You don’t find the key in this node either, so you must go to the next child. Here, because 89 is greater than 83 but less than 91, you go to child 2. This time, you find the specified item in the 85-89 node.

Insertion

New data items are always inserted in leaves, which are on the bottom row of the tree. If items were inserted in nodes with children, the number of children would need to be changed to maintain the structure of the tree, which stipulates that there should be one more child than data items in a node.

Insertion into a 2-3-4 tree is sometimes quite easy and sometimes rather complicated. In any case, the process begins by searching for the appropriate leaf node.

If there is space in all the nodes encountered during the search, insertion is easy. When the appropriate leaf node is reached, the new data item is simply inserted into it. [Figure 9-3](#) shows a data item with key 46 being inserted into the 2-3-4 tree from [Figure 9-1](#). The path to the leaf node where 46 belongs is shown.

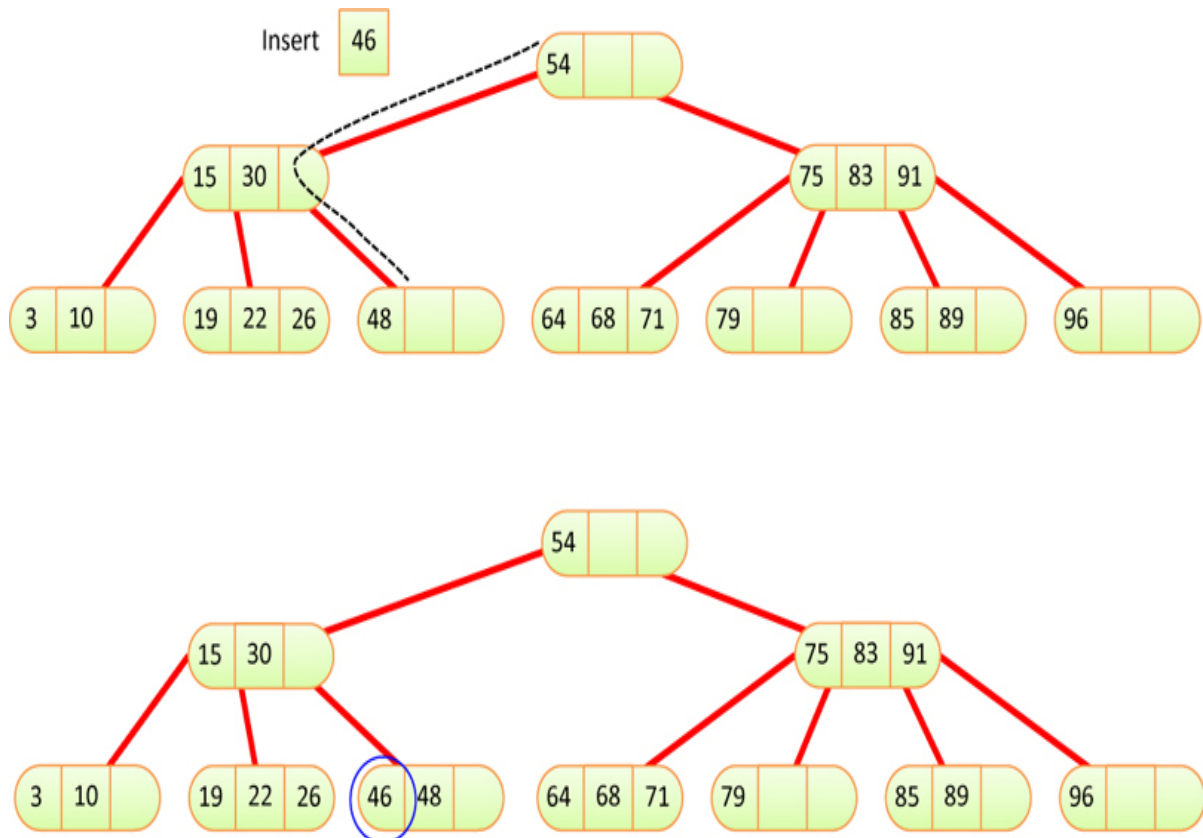


Figure 9-3 *Insertion with no splits*

Insertion may involve moving one or two other items in a node so that the keys will be in the correct order after the new item is inserted. In this example, the 48 had to be shifted right to make room for the 46.

Node Splits

Insertion becomes more complicated if a full node is encountered on the path down to the insertion point. A *full node* is one that contains three data items. When this happens, the node must be *split*. The splitting process keeps the tree balanced. The kind of 2-3-4 tree we're discussing here is often called a *top-down 2-3-4 tree* because nodes are split on the way down to the insertion point.

Here's what happens when the insertion process splits a nonroot node. (We examine splitting the root later.)

- A new, empty node is created. It's a sibling of the node being split and will be placed to its right.
- Data item 2 is moved into the new node.
- Data item 1 is moved into the parent of the node being split.
- Data item 0 remains where it is in the node being split.
- The rightmost two children are disconnected from the node being split and connected to the new node.

Figure 9-4 shows an example of a node split. Another way of describing a node split is to say that a 4-node has been transformed into two 2-nodes.

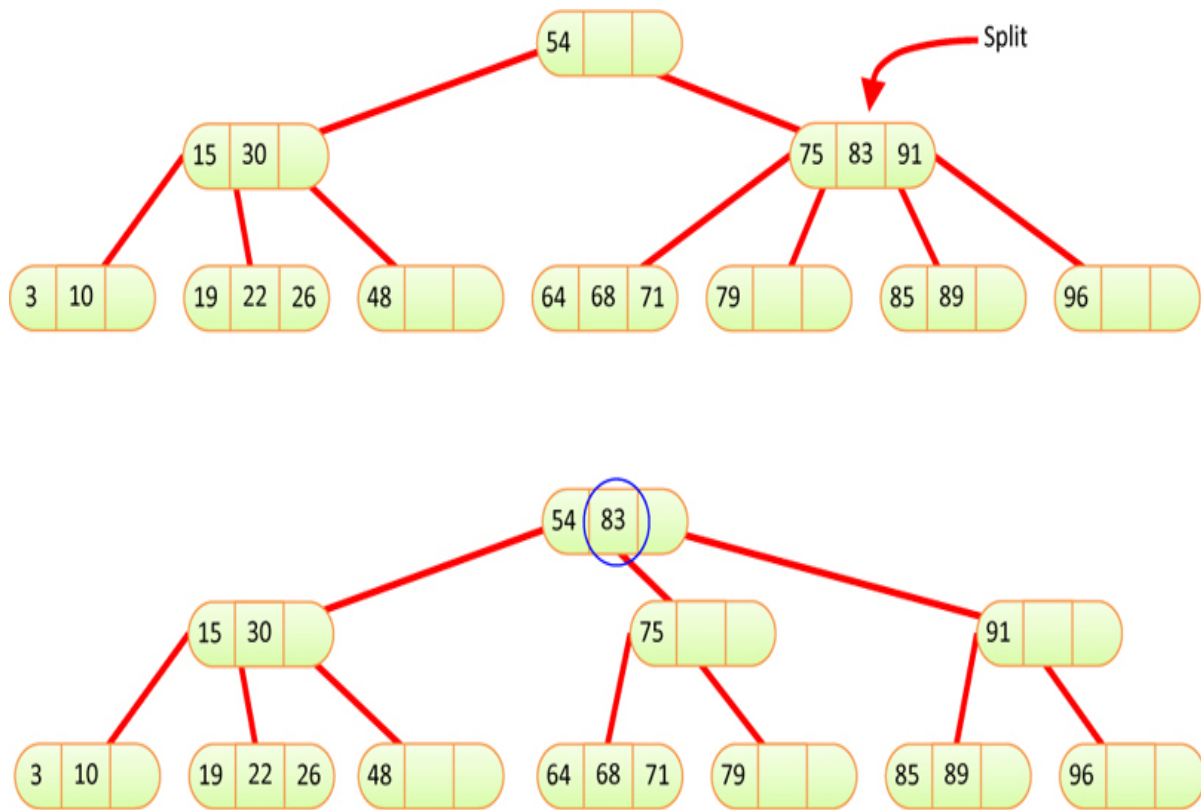


Figure 9-4 *Splitting a nonroot node*

Notice that the effect of the node split is to move data up or to the right. Key 1 from the split node moved up to the parent as circled in the figure. The 75 and 91 keys go into their own nodes as the children on either side of the promoted key. This rearrangement keeps the tree balanced.

Here, the insertion required only one node split, but more than one full node may be encountered along the path to the insertion point. When this is the case, there will be multiple splits.

Full nodes can occur anywhere in the tree. When the full node is a leaf, it has no children, such as nodes 19-22-26 and 64-68-71 in [Figure 9-4](#). If it's an internal node, then it must have four children, as node 75-83-91 did before it was split.

Splitting the Root

When a full root node is encountered at the beginning of the search for the insertion point, the resulting split is slightly more complicated: